

C 语言八股文

一、指针体系

1. 一级指针和二级指针的区别？

指针的本质：指针是一个变量，存储的是内存地址。指针的大小在 32 位系统下为 4 字节，64 位系统下为 8 字节。

一级指针：存储普通变量的地址，通过 `*` 解引用访问变量的值。

二级指针：存储一级指针的地址，通过 `**` 两次解引用访问最终变量的值。常用于函数中修改指针本身的值（如动态内存分配、链表节点操作）。

```
#include <stdio.h>
#include <stdlib.h>

void changeValue(int *p) {    // 一级指针：修改指针指向的值
    *p = 100;
}

void changePointer(int **pp) { // 二级指针：修改指针本身的指向
    int *newP = (int *)malloc(sizeof(int));
    *newP = 200;
    *pp = newP;
}

int main() {
    int a = 10;
    int *p = &a;
    int **pp = &p;

    printf("a = %d, *p = %d, **pp = %d\n", a, *p, **pp);

    changeValue(p);
    printf("changeValue后: a = %d\n", a);

    changePointer(pp);
    printf("changePointer后: *p = %d\n", *p);

    free(p);
    return 0;
}
```

```
/*
运行结果：
a = 10, *p = 10, **pp = 10
changeValue后: a = 100
changePointer后: *p = 200
*/
```

延伸：三级指针的应用场景（如动态二维数组的创建）：

```
// 动态创建 m×n 的二维数组
int **create2DArray(int m, int n) {
    int **arr = (int **)malloc(m * sizeof(int *));
    for (int i = 0; i < m; i++) {
        arr[i] = (int *)malloc(n * sizeof(int));
    }
    return arr;
}
```

2. 指针与数组的区别？

对比项	数组	指针
本质	一组连续内存空间的集合	存储地址的变量
赋值	数组名是常量，不能被赋值	指针变量可以重新指向
sizeof	返回整个数组大小	返回指针本身大小（4/8字节）
传参	退化为指针，丢失长度信息	本身就是指针

```
#include <stdio.h>

void printArray(int arr[], int size) { // 数组传参退化为指针
    printf("函数内 sizeof(arr) = %lu\n", sizeof(arr)); // 8（指针大小）
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr;

    printf("sizeof(arr) = %lu\n", sizeof(arr)); // 整个数组：5*4=20
    printf("sizeof(p) = %lu\n", sizeof(p)); // 指针：8

    // 指针运算与数组下标等价
    printf("arr[2] = %d, *(p+2) = %d\n", arr[2], *(p + 2));

    // 指针可以移动，数组名不能
    p++;
    printf("p++后 *p = %d\n", *p); // 2

    printArray(arr, 5);

    return 0;
}
```

```

/*
运行结果：
sizeof(arr) = 20
sizeof(p)    = 8
arr[2] = 3, *(p+2) = 3
p++后 *p = 2
函数内 sizeof(arr) = 8
*/

```

延伸：数组名在什么情况下不退化为指针？

- `sizeof(数组名)`：返回整个数组大小
- `&数组名`：返回整个数组的地址，类型为 `int (*)[5]`

```

int arr[5] = {1,2,3,4,5};
printf("%lu\n", sizeof(arr));    // 20
printf("%p\n", arr);            // 首元素地址
printf("%p\n", &arr);           // 整个数组地址（值相同，类型不同）
printf("%p\n", arr + 1);        // 偏移4字节（一个int）
printf("%p\n", &arr + 1);       // 偏移20字节（整个数组）

```

3. 指针数组和数组指针的区别？

记忆口诀：从右往左读，`[]` 优先级高于 `*`。

- **指针数组：** `int *p[5]` —— 先结合 `[]`，是数组，每个元素是 `int*` 指针。
- **数组指针：** `int (*p)[5]` —— 先结合 `*`，是指针，指向一个含 5 个 `int` 的数组。

```

#include <stdio.h>

int main() {
    // 指针数组：数组里存指针，常用于存储多个字符串
    const char *strArr[3] = {"Hello", "world", "C"};
    for (int i = 0; i < 3; i++)
        printf("%s ", strArr[i]);
    printf("\n");

    // 数组指针：指向数组的指针，常用于二维数组传参
    int arr[2][3] = {{1,2,3},{4,5,6}};
    int (*p)[3] = arr; // p指向含3个int的数组

    printf("arr[1][2] = %d, p[1][2] = %d\n", arr[1][2], p[1][2]);
    printf("*(p[1]+2) = %d\n", *(p[1] + 2)); // 等价写法

    return 0;
}

```

```
/*
运行结果：
Hello world C
arr[1][2] = 6, p[1][2] = 6
*(p[1]+2) = 6
*/
```

延伸：函数指针数组（跳转表）的应用：

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main() {
    // 函数指针数组
    int (*ops[3])(int, int) = {add, sub, mul};

    printf("add: %d\n", ops[0](3, 5));    // 8
    printf("sub: %d\n", ops[1](10, 3));   // 7
    printf("mul: %d\n", ops[2](4, 6));    // 24

    return 0;
}
```

4. 指针与字符串的关系？

字符串在 C 语言中以 `\0` 结尾的字符数组表示。指针可以指向字符串常量或字符数组。

```
#include <stdio.h>
#include <string.h>

int main() {
    // 方式1: 字符数组（可修改）
    char str1[] = "Hello";
    str1[0] = 'h'; // 合法
    printf("str1: %s\n", str1);

    // 方式2: 指针指向字符串常量（不可修改）
    const char *str2 = "Hello";
    // str2[0] = 'h'; // 非法！字符串常量存储在只读数据段

    // 字符串处理函数
    printf("strlen: %lu\n", strlen(str1));    // 5
    printf("strcmp: %d\n", strcmp(str1, str2)); // 0（相等）

    // 指针遍历字符串
    const char *p = str2;
    while (*p) {
        printf("%c", *p);
        p++;
    }
    printf("\n");
}
```

```
return 0;
}
```

延伸：字符串常量的存储位置：

- 字符串常量存储在只读数据段 (.rodata)
- 相同内容的字符串常量可能被编译器合并（字符串池）

```
const char *s1 = "Hello";
const char *s2 = "Hello";
printf("%p\n", s1); // 相同地址
printf("%p\n", s2); // 相同地址（编译器优化）
```

二、内存管理

5. 栈和堆的区别？

对比项	栈 (Stack)	堆 (Heap)
分配方式	编译器自动分配	程序员手动 malloc/free
生命周期	函数调用时创建，返回时销毁	手动管理，直到 free
大小限制	较小（通常 1-8MB）	较大（取决于虚拟内存）
分配速度	快（移动栈指针）	慢（需要搜索空闲块）
碎片问题	无	有（频繁分配释放产生碎片）
存储内容	局部变量、函数参数、返回地址	动态分配的内存

```
#include <stdio.h>
#include <stdlib.h>

int *createArray(int n) {
    // 堆上分配，函数返回后内存仍然有效
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) arr[i] = i;
    return arr;
}

int* badExample(int n) {
    // 栈上分配，函数返回后内存失效！
    int arr[100];
    return arr; // 警告：返回局部变量的地址
}

int main() {
    int a = 10; // 栈上
    int *p = createArray(5); // 堆上

    for (int i = 0; i < 5; i++)
        printf("%d ", p[i]);
}
```

```
printf("\n");

free(p); // 必须手动释放，否则内存泄漏
return 0;
}
```

延伸：内存分区全景图：



6. malloc 和 free 的使用注意事项？

核心原则：谁分配，谁释放；分配后检查 NULL；释放后置 NULL。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // 1. 分配后必须检查 NULL
    int *p = (int *)malloc(5 * sizeof(int));
    if (p == NULL) {
        printf("内存分配失败\n");
        return -1;
    }

    // 2. 初始化内存 (malloc 不初始化, calloc 会初始化为0)
    memset(p, 0, 5 * sizeof(int));
    // 或者用 calloc: int *p = (int *)calloc(5, sizeof(int));

    // 3. 使用内存
    for (int i = 0; i < 5; i++) p[i] = i * 10;

    // 4. 释放后置 NULL，防止野指针
    free(p);
    p = NULL;
}
```

```
// 5. 常见错误示例
// free(p);      // 危险! 重复 free 会导致崩溃
// int *q = p;    // 危险! p 已释放, q 成为野指针

return 0;
}
```

常见内存错误及检测工具:

错误类型	描述	检测方法
内存泄漏	malloc 后未 free	Valgrind、AddressSanitizer
野指针	free 后继续使用	释放后置 NULL
越界访问	访问超出分配范围	ASan、Valgrind
重复释放	同一内存 free 两次	释放后置 NULL
使用未初始化内存	malloc 后未初始化就使用	Valgrind

延伸: 内存泄漏检测工具 Valgrind 使用:

```
# 编译时加 -g 调试信息
gcc -g main.c -o main

# 使用 valgrind 检测
valgrind --leak-check=full ./main
```

7. 内存对齐是什么? 为什么需要内存对齐?

内存对齐规则:

- 1. 每个成员变量的起始地址必须是其大小的整数倍
- 2. 结构体总大小必须是最大成员大小的整数倍
- 3. 可以使用 `#pragma pack(n)` 修改对齐方式

为什么需要对齐:

- **性能:** CPU 按字长读取内存, 对齐后一次读取即可, 不对齐需要多次读取
- **硬件限制:** 某些架构 (如 ARM) 要求严格对齐, 否则触发异常

```
#include <stdio.h>

struct A {
    char a;    // 1字节, 偏移0
    // 填充3字节
    int b;     // 4字节, 偏移4
    char c;    // 1字节, 偏移8
    // 填充3字节
};           // 总大小: 12字节

struct B {
```

```
char a;    // 1字节
char c;    // 1字节
// 填充2字节
int b;     // 4字节
};         // 总大小: 8字节 (优化排列)

#pragma pack(1) // 取消对齐
struct C {
    char a;
    int b;
    char c;
};         // 总大小: 6字节
#pragma pack() // 恢复默认对齐

int main() {
    printf("sizeof(struct A) = %lu\n", sizeof(struct A)); // 12
    printf("sizeof(struct B) = %lu\n", sizeof(struct B)); // 8
    printf("sizeof(struct C) = %lu\n", sizeof(struct C)); // 6

    return 0;
}
```

延伸：offsetof 宏获取成员偏移量：

```
#include <stddef.h>

printf("a偏移: %lu\n", offsetof(struct A, a)); // 0
printf("b偏移: %lu\n", offsetof(struct A, b)); // 4
printf("c偏移: %lu\n", offsetof(struct A, c)); // 8
```

三、高级数据类型

8. 结构体和共用体的区别？

对比项	结构体 (struct)	共用体 (union)
内存分配	每个成员独立内存	所有成员共享同一内存
大小	≥ 所有成员大小之和 (考虑对齐)	= 最大成员大小
同时有效	所有成员同时有效	同一时刻只有一个成员有效
应用场景	存储不同类型的相关数据	节省内存、类型转换

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float score;
};

union Data {
```



```

    int i;
    float f;
    char c;
};

// 共用体应用：判断系统大小端
void checkEndian() {
    union {
        int i;
        char c;
    } test;
    test.i = 1;
    if (test.c == 1)
        printf("小端模式\n");
    else
        printf("大端模式\n");
}

int main() {
    printf("sizeof(struct Student) = %lu\n", sizeof(struct Student)); // 28
    printf("sizeof(union Data)      = %lu\n", sizeof(union Data));    // 4

    union Data d;
    d.i = 65;
    printf("d.i = %d, d.c = %c\n", d.i, d.c); // 共用同一内存

    d.f = 3.14;
    printf("d.f = %f, d.i = %d\n", d.f, d.i); // i的值被覆盖

    checkEndian();

    return 0;
}

```

```

/*
运行结果：
sizeof(struct Student) = 28
sizeof(union Data)      = 4
d.i = 65, d.c = A
d.f = 3.140000, d.i = 1078523331
小端模式
*/

```

延伸：结构体位域（节省内存）：

```

struct Flags {
    unsigned int flag1 : 1; // 占1位
    unsigned int flag2 : 1; // 占1位
    unsigned int type  : 4; // 占4位
    unsigned int reserved : 26; // 保留位
}; // 总共4字节

int main() {
    struct Flags f;

```

```
f.flag1 = 1;
f.flag2 = 0;
f.type = 0xA;
printf("sizeof(Flags) = %lu\n", sizeof(f)); // 4
return 0;
}
```

9. 枚举的作用和使用场景？

枚举用于定义一组命名的整数常量，提高代码可读性。

```
#include <stdio.h>

// 定义枚举
enum Color {
    RED = 0,
    GREEN = 1,
    BLUE = 2
};

// 状态机应用
enum State {
    IDLE,
    RUNNING,
    PAUSED,
    STOPPED
};

int main() {
    enum Color c = GREEN;
    printf("Color: %d\n", c); // 1

    // 枚举在 switch 中的应用
    enum State state = RUNNING;
    switch (state) {
        case IDLE:    printf("Idle\n"); break;
        case RUNNING: printf("Running\n"); break;
        case PAUSED:  printf("Paused\n"); break;
        case STOPPED: printf("Stopped\n"); break;
    }

    return 0;
}
```

延伸：枚举与宏定义的对比：

对比项	枚举	#define
类型检查	有（编译器检查）	无（文本替换）
调试	可以显示枚举名	只显示数值
作用域	遵循作用域规则	全局有效

对比项	枚举	#define
自动赋值	可以自动递增	需要手动指定

10. 函数指针和指针函数的区别？

记忆方法：从右往左读，看 * 和 () 谁先结合。

- **指针函数**：`int *func()` —— `()` 优先级高，是函数，返回值是 `int*`。
- **函数指针**：`int (*func)(int, int)` —— `*` 先结合，是指针，指向函数。

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

// 函数指针作为参数，实现回调
int calculate(int (*op)(int, int), int a, int b) {
    return op(a, b);
}

// 函数指针数组（跳转表）
typedef int (*OpFunc)(int, int);

int main() {
    // 定义函数指针
    int (*pFunc)(int, int) = add;
    printf("add: %d\n", pFunc(3, 5));

    pFunc = sub;
    printf("sub: %d\n", pFunc(10, 3));

    // 作为回调使用
    printf("calculate(add): %d\n", calculate(add, 3, 5));
    printf("calculate(sub): %d\n", calculate(sub, 10, 3));

    // 函数指针数组
    OpFunc ops[] = {add, sub};
    printf("ops[0]: %d\n", ops[0](1, 2));
    printf("ops[1]: %d\n", ops[1](5, 3));

    return 0;
}
```

```
/*
运行结果:
add: 8
sub: 7
calculate(add): 8
calculate(sub): 7
ops[0]: 3
ops[1]: 2
*/
```

延伸：函数指针在策略模式中的应用：

```
// 排序策略
typedef int (*CompareFunc)(const void *, const void *);

int compareAsc(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}

int compareDesc(const void *a, const void *b) {
    return *(int *)b - *(int *)a;
}

void sort(int arr[], int n, CompareFunc cmp) {
    // 使用 qsort, 传入比较函数
    qsort(arr, n, sizeof(int), cmp);
}
```

11. 宏定义和 typedef 的区别？

对比项	#define	typedef
处理阶段	预处理阶段（文本替换）	编译阶段（类型别名）
类型检查	无	有
作用域	从定义处到文件结束（或 #undef）	遵循作用域规则
功能	可以定义常量、宏函数	只能定义类型别名

```
#include <stdio.h>

#define INT_PTR int *
typedef int * int_ptr;

// 宏函数
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))

int main() {
    INT_PTR a, b;      // 展开为 int *a, b; → a是指针, b是int
    int_ptr c, d;      // c和d都是 int* 类型
```

```

printf("sizeof(a)=%lu, sizeof(b)=%lu\n", sizeof(a), sizeof(b));
printf("sizeof(c)=%lu, sizeof(d)=%lu\n", sizeof(c), sizeof(d));

// 宏函数注意事项：必须加括号
int x = 3, y = 5;
printf("MAX: %d\n", MAX(x, y));          // 5
printf("SQUARE: %d\n", SQUARE(x + 1)); // 16 (不是7)

// #define 可以定义常量
#define PI 3.14159
printf("PI = %f\n", PI);

return 0;
}

```

```

/*
运行结果：
sizeof(a)=8, sizeof(b)=4
sizeof(c)=8, sizeof(d)=8
MAX: 5
SQUARE: 16
PI = 3.141590
*/

```

延伸：条件编译（跨平台开发）：

```

#include <stdio.h>

#ifdef _WIN32
    #define OS_NAME "windows"
#elif defined(__linux__)
    #define OS_NAME "Linux"
#elif defined(__APPLE__)
    #define OS_NAME "macOS"
#else
    #define OS_NAME "Unknown"
#endif

#ifndef DEBUG
    #define LOG(msg)
#else
    #define LOG(msg) printf("[DEBUG] %s\n", msg)
#endif

int main() {
    printf("OS: %s\n", OS_NAME);
    LOG("This is a debug message");
    return 0;
}

```

四、数据结构与文件 I/O

12. 如何实现单向链表?

链表由节点组成，每个节点包含数据域和指针域，通过指针串联。

核心操作：

- 头插法：新节点插入头部， $O(1)$
- 尾插法：新节点插入尾部， $O(n)$ 或 $O(1)$ (维护尾指针)
- 删除节点：找到前驱节点，修改指针
- 反转链表：迭代或递归

```
#include <stdio.h>
#include <stdlib.h>

// 节点定义
typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 尾插法创建链表
Node* createList(int arr[], int n) {
    Node *head = NULL, *tail = NULL;
    for (int i = 0; i < n; i++) {
        Node *newNode = (Node *)malloc(sizeof(Node));
        newNode->data = arr[i];
        newNode->next = NULL;
        if (head == NULL) head = tail = newNode;
        else { tail->next = newNode; tail = newNode; }
    }
    return head;
}

// 头插法
Node* insertHead(Node *head, int val) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = head;
    return newNode;
}

// 删除指定值的节点
Node* deleteNode(Node *head, int val) {
    Node dummy; dummy.next = head;
    Node *prev = &dummy, *cur = head;
    while (cur) {
        if (cur->data == val) {
            prev->next = cur->next;
            free(cur);
            cur = prev->next;
        } else { prev = cur; cur = cur->next; }
    }
}
```

```

        return dummy.next;
    }

// 反转链表 (迭代)
Node* reverseList(Node *head) {
    Node *prev = NULL, *cur = head, *next = NULL;
    while (cur) {
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    return prev;
}

// 遍历打印
void printList(Node *head) {
    while (head) { printf("%d -> ", head->data); head = head->next; }
    printf("NULL\n");
}

// 释放链表
void freeList(Node *head) {
    while (head) {
        Node *tmp = head;
        head = head->next;
        free(tmp);
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    Node *head = createList(arr, 5);
    printList(head);           // 1 -> 2 -> 3 -> 4 -> 5 -> NULL

    head = insertHead(head, 0);
    printList(head);           // 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL

    head = deleteNode(head, 3);
    printList(head);           // 0 -> 1 -> 2 -> 4 -> 5 -> NULL

    head = reverseList(head);
    printList(head);           // 5 -> 4 -> 2 -> 1 -> 0 -> NULL

    freeList(head);
    return 0;
}

```

```

/*
运行结果:
1 -> 2 -> 3 -> 4 -> 5 -> NULL
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
0 -> 1 -> 2 -> 4 -> 5 -> NULL
5 -> 4 -> 2 -> 1 -> 0 -> NULL
*/

```

延伸：快慢指针找链表中间节点、判断环：

```
// 快慢指针找中间节点
Node* findMiddle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// 判断链表是否有环
int hasCycle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;
    }
    return 0;
}
```

13. 文件 I/O 的基本操作？

C 语言通过 FILE* 指针操作文件，核心函数：

函数	功能
fopen	打开文件
fclose	关闭文件
fread/fwrite	二进制读写
fscanf/fprintf	格式化读写
fgets/fputs	字符串读写
fseek/ftell/rewind	文件定位

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 1. 写文件
    FILE *fp = fopen("data.txt", "w");
    if (!fp) { perror("fopen"); return -1; }

    fprintf(fp, "Hello, File!\n");
    fprintf(fp, "Line 2: %d\n", 100);
    fclose(fp);

    // 2. 读文件（逐行）
}
```



```

fp = fopen("data.txt", "r");
char buf[100];
while (fgets(buf, sizeof(buf), fp))
    printf("%s", buf);
fclose(fp);

// 3. 二进制读写
fp = fopen("binary.dat", "wb");
int arr[] = {1, 2, 3, 4, 5};
fwrite(arr, sizeof(int), 5, fp);
fclose(fp);

fp = fopen("binary.dat", "rb");
int readArr[5];
fread(readArr, sizeof(int), 5, fp);
fclose(fp);

// 4. 文件定位
fp = fopen("data.txt", "r");
fseek(fp, 0, SEEK_END);
long size = ftell(fp);
printf("File size: %ld bytes\n", size);
rewind(fp);
fclose(fp);

return 0;
}

```

延伸：文件打开模式：

模式	含义
"r"	只读，文件必须存在
"w"	只写，文件不存在则创建，存在则清空
"a"	追加，文件不存在则创建
"r+"	读写，文件必须存在
"w+"	读写，文件不存在则创建，存在则清空
"a+"	读追加，文件不存在则创建
"rb"/"wb"	二进制模式（Windows 下区分文本/二进制）

14. 预处理指令有哪些？

```

#include <stdio.h>

// 1. 宏定义
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))

```

```
// 2. 条件编译
#ifdef DEBUG
    #define LOG(msg) printf("[DEBUG] %s\n", msg)
#else
    #define LOG(msg)
#endif

// 3. 文件包含
// #include <stdio.h> // 系统头文件
// #include "myheader.h" // 自定义头文件

// 4. 其他预处理指令
// #pragma once // 防止头文件重复包含（非标准，但广泛支持）
// #error "This is an error" // 产生编译错误
// #warning "This is a warning" // 产生编译警告

int main() {
    printf("PI = %f\n", PI);
    printf("MAX(3, 5) = %d\n", MAX(3, 5));
    LOG("Debug mode enabled");
    return 0;
}
```

延伸：防止头文件重复包含的两种方法：

```
// 方法1: 条件编译（标准）
#ifndef MYHEADER_H
#define MYHEADER_H
// 头文件内容
#endif

// 方法2: #pragma once（非标准，但主流编译器支持）
#pragma once
// 头文件内容
```

五、综合应用

15. 贪吃蛇项目中的关键技术点？

贪吃蛇项目综合运用了 C 语言的核心知识点：

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>

// 1. 结构体定义蛇节点
typedef struct Node {
    int x, y;
    struct Node *next;
} SnakeNode;

// 2. 全局变量
```

```

SnakeNode *head = NULL, *tail = NULL;
int foodX, foodY;
int direction = 1; // 1:上 2:下 3:左 4:右
int score = 0;

// 3. 链表操作：蛇的移动（头插法 + 尾删法）
void moveSnake() {
    // 计算新头部位置
    int newX = head->x, newY = head->y;
    switch (direction) {
        case 1: newY--; break; // 上
        case 2: newY++; break; // 下
        case 3: newX--; break; // 左
        case 4: newX++; break; // 右
    }

    // 检查碰撞
    if (newX < 0 || newX >= 20 || newY < 0 || newY >= 20) {
        printf("Game Over! Score: %d\n", score);
        exit(0);
    }

    // 头插法添加新节点
    SnakeNode *newNode = (SnakeNode *)malloc(sizeof(SnakeNode));
    newNode->x = newX;
    newNode->y = newY;
    newNode->next = head;
    head = newNode;

    // 检查是否吃到食物
    if (newX == foodX && newY == foodY) {
        score++;
        generateFood();
    } else {
        // 没吃到食物，删除尾部
        SnakeNode *prev = head;
        while (prev->next != tail) prev = prev->next;
        free(tail);
        tail = prev;
        tail->next = NULL;
    }
}

// 4. 文件 I/O: 保存最高分
void saveHighScore(int score) {
    FILE *fp = fopen("highscore.txt", "w");
    fprintf(fp, "%d", score);
    fclose(fp);
}

int loadHighScore() {
    FILE *fp = fopen("highscore.txt", "r");
    int hs = 0;
    if (fp) {
        fscanf(fp, "%d", &hs);
        fclose(fp);
    }
}

```

```

    }
    return hs;
}

int main() {
    // 游戏主循环
    while (1) {
        if (_kbhit()) {
            char key = _getch();
            switch (key) {
                case 'w': direction = 1; break;
                case 's': direction = 2; break;
                case 'a': direction = 3; break;
                case 'd': direction = 4; break;
            }
        }
        moveSnake();
        drawGame();
        sleep(100); // 控制速度
    }
    return 0;
}

```

项目涉及知识点总结：

- **链表**：蛇身用链表表示，移动时头插尾删
- **结构体**：定义蛇节点、食物、游戏状态
- **指针**：链表操作、内存管理
- **文件 I/O**：保存/加载最高分
- **宏定义**：游戏常量（地图大小、初始速度）
- **函数封装**：模块化设计（移动、绘制、碰撞检测）

六、常见面试题汇总

16. 野指针和悬空指针的区别？

- **野指针**：未初始化的指针，指向随机地址。
- **悬空指针**：指向已释放内存的指针。

```

int *p;           // 野指针（未初始化）
int *q = malloc(sizeof(int));
free(q);
// q 现在是悬空指针
q = NULL;        // 正确做法：释放后置 NULL

```

17. const 和 #define 的区别？

对比项	const	#define
类型检查	有	无
作用域	遵循作用域	全局
调试	可以调试	预处理后消失
内存	可能分配内存	不分配（文本替换）

18. static 关键字的作用？

1. **修饰局部变量**：延长生命周期到程序结束，但作用域不变
2. **修饰全局变量/函数**：限制作用域为当前文件（内部链接）
3. **C++ 中修饰类成员**：属于类而非对象

```
void func() {
    static int count = 0; // 只初始化一次，生命周期到程序结束
    count++;
    printf("count = %d\n", count);
}
```

19. 如何防止内存泄漏？

1. **配对使用**：malloc/free、new/delete 必须配对
2. **释放后置 NULL**：防止悬空指针
3. **使用智能指针（C++）**：自动管理内存
4. **使用检测工具**：Valgrind、AddressSanitizer
5. **代码审查**：检查所有返回路径是否都释放了内存

20. 大端和小端的区别？

- **大端**：高字节存储在低地址（网络字节序）
- **小端**：低字节存储在低地址（x86 架构）

```
#include <stdio.h>

int main() {
    int x = 0x12345678;
    char *p = (char *)&x;

    if (*p == 0x78)
        printf("小端模式\n");
    else if (*p == 0x12)
        printf("大端模式\n");

    return 0;
}
```

附录：C 语言学习路线图

